

JFXMaze Detailed Project Report

Programming II Final Project - JavaFX Desktop Application

Field	Value
Project Title	JFXMaze
Displayed Game Name	NEON MAZE
Student Name	Miar Albarghouth
Student ID	24040102053
GitHub Repository	https://github.com/miaralbarghouth/JFXMaze_neonmaze
Technologies	Java 17, JavaFX 21, Maven

1. Project Overview

JFXMaze is a JavaFX-based desktop maze game presented to the user as NEON MAZE. The application combines algorithmic maze generation, user authentication, local file processing, score tracking, and a visually styled neon interface. The player logs in, starts a new maze, collects points, uses optional hints, and attempts to reach the exit as efficiently as possible.

2. Project Goals

- Implement a complete JavaFX visual application using Java and Maven.
- Provide user registration, login, and logout features.
- Save application data on the local drive using file processing.
- Create a playable and visually clear maze game with scoring and leaderboard support.
- Demonstrate graph/search algorithms through DFS maze generation and BFS pathfinding.

3. Main Features

- Authentication: users can register, log in, and log out safely.
- Password storage: account passwords are stored as salted SHA-256 hashes.
- File processing: account data and leaderboard records are stored locally.
- Maze generation: each game creates a new random maze using a depth-first search approach.
- Hint system: the game can calculate a path to the exit using breadth-first search.
- Leaderboard: completed games can be saved and ranked by score weight.

- Two-language UI: English is the default language, and Turkish can be selected in settings.
- Audio controls: background music and sound effects are supported.

4. System Design and Package Structure

The project is organized into packages that separate the model, game logic, and user interface:

Field	Value
com.mazegame	Application entry point classes: Main and MazeApp.
com.mazegame.model	Data models such as Maze and ScoreRecord.
com.mazegame.logic	Managers for authentication, app data paths, maze generation, leaderboard, music, and sound.
com.mazegame.ui	Screen management, language handling, and JavaFX navigation.
com.mazegame.ui.components	Reusable game view component containing the canvas game loop.
src/main/resources	CSS styling and audio resources.

5. Authentication Implementation

Authentication is implemented through AuthManager and ScreenManager. AuthManager loads and saves user accounts, registers new users, validates login attempts, changes passwords, renames users, and deletes accounts. The default test account is mk / mk. Passwords are not saved as plain text; each password is encoded with a random salt and SHA-256 hashing before being written to local storage.

6. File Processing

The application stores runtime data in a .jfxmaze folder under the user home directory. This keeps generated data outside the source-code project folder and prevents runtime files from being submitted or uploaded to GitHub by mistake.

Field	Value
users.properties	Stores usernames and hashed password values.
leaderboard.dat	Stores serialized ScoreRecord objects for leaderboard history.

7. Maze Generation and Game Logic

The maze is represented by the Maze model as a grid of cells. MazeManager generates a random maze using a depth-first search style algorithm and verifies playable paths. The player starts near the upper-left side of the maze and must reach the exit near the lower-right side. Collectibles

increase the score, while steps and time are tracked during gameplay.

8. BFS Hint System

When the player presses H, the game calculates a hint path using breadth-first search. BFS is suitable for this feature because it explores the maze level by level and can find a short route from the player current position to the exit. The result is displayed visually on the canvas.

9. User Interface

The interface is built with JavaFX screens managed by ScreenManager. The game screen uses a Canvas in GameView for custom drawing of walls, paths, collectibles, the player, hints, and the exit. The visual design uses a dark neon style supported by CSS and simple animation transitions.

10. Screenshots

The following screenshots show the successful working of the application and its required features.

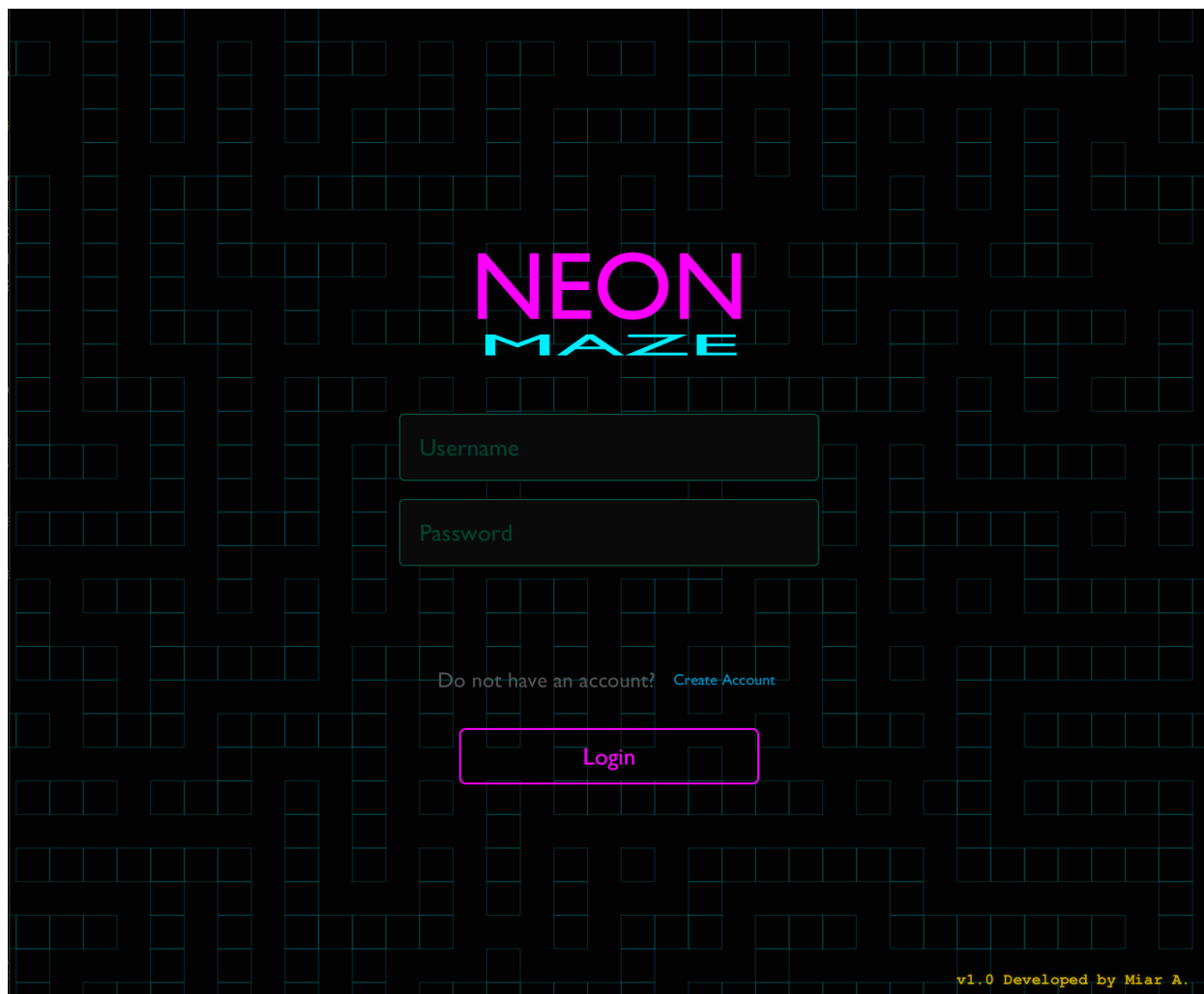


Figure 1. Login screen with username and password fields.

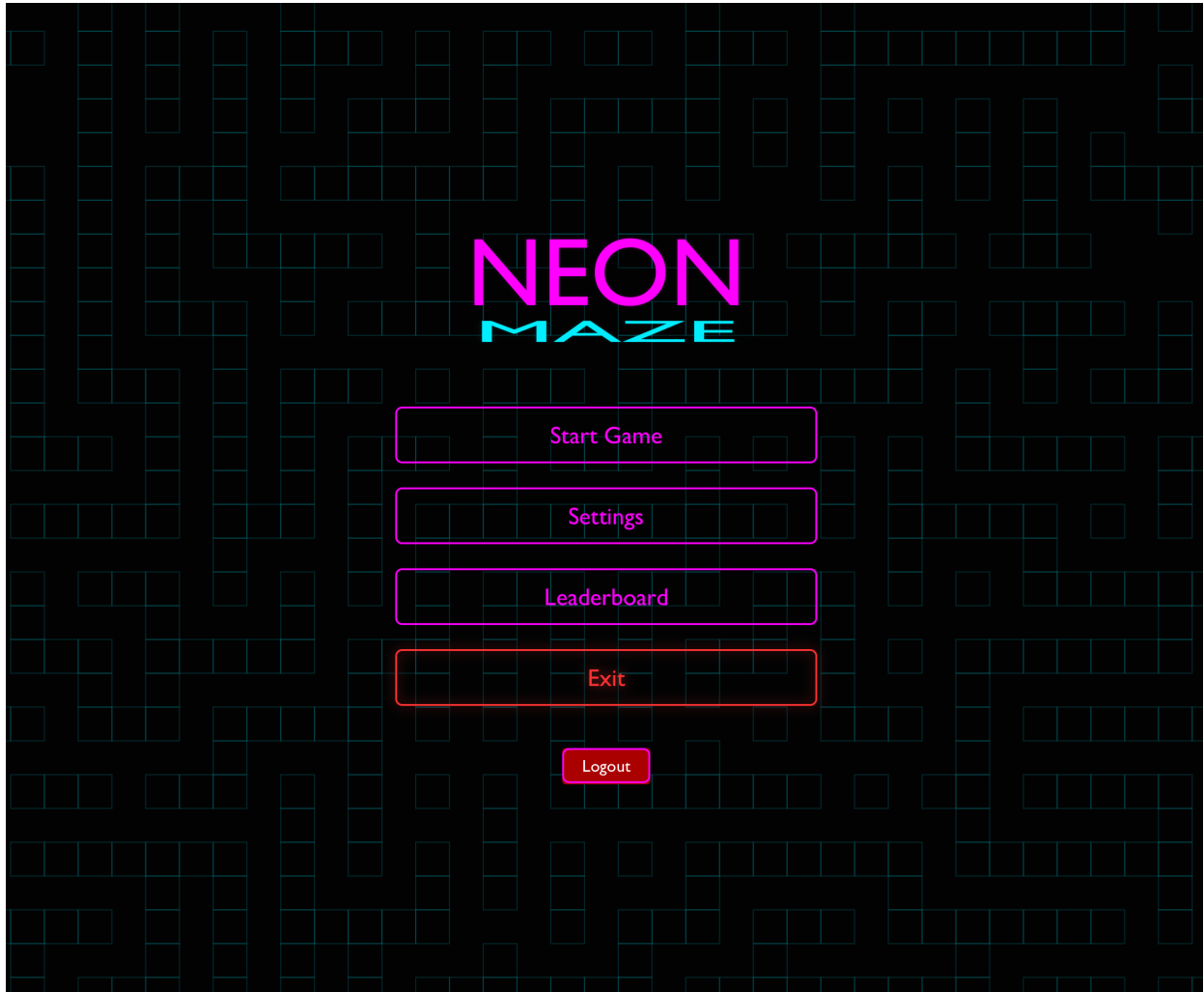


Figure 2. Main menu after authentication, including game, settings, leaderboard, and logout actions.

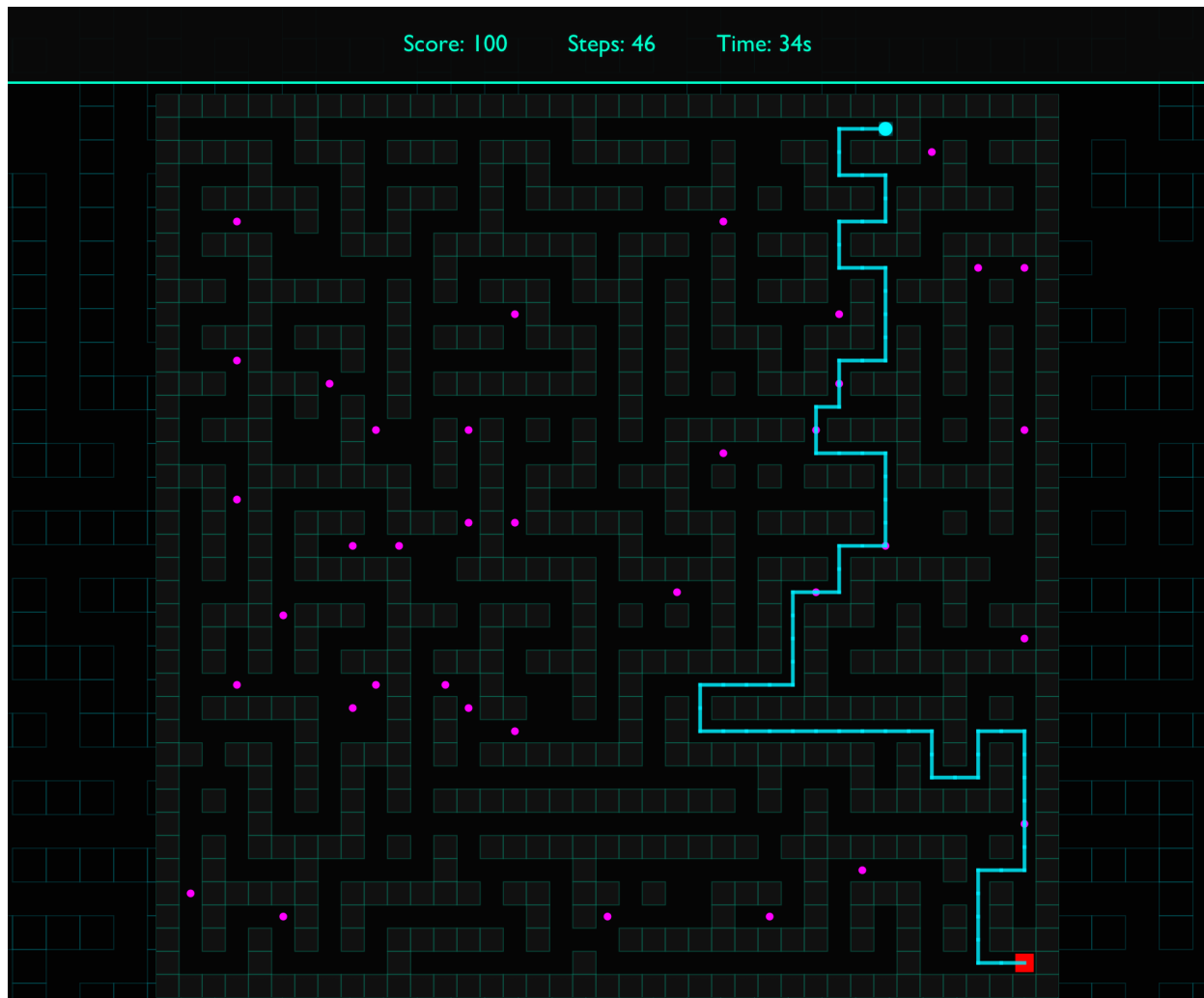


Figure 3. Gameplay screen showing the generated maze, score, timer, collectibles, player, exit, and BFS hint path.



Figure 4. Settings screen with audio controls, Turkish/English language options, and controls information.

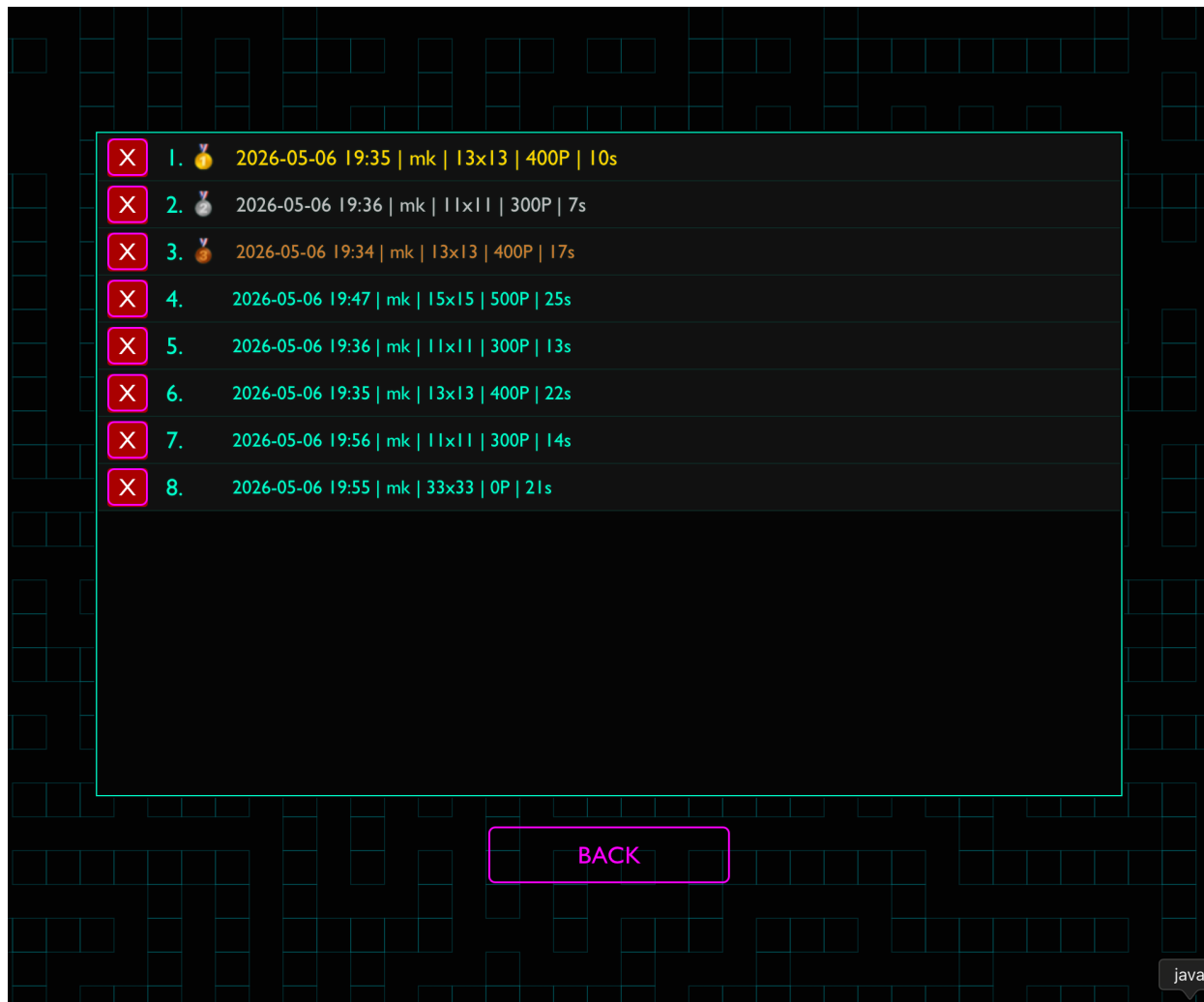


Figure 5. Leaderboard screen showing saved local score records.

11. Distinctive Aspects

- The game does not rely on a fixed map; every session can generate a new maze.
- The project combines both DFS and BFS algorithms in visible gameplay features.
- The application includes authentication and persistent local data storage, not only gameplay.
- The leaderboard ranking uses a weighted score based on points, maze size, and completion time.
- Runtime data is stored outside the repository folder, which keeps the submitted project clean.

12. Installation and Running

The project can be downloaded from GitHub and run with Maven:

```
git clone https://github.com/miaralbarghouth/JFXMaze_neonmaze.git
cd JFXMaze_neonmaze
```

```
mvn javafx:run
```

13. Conclusion

JFXMaze satisfies the final project requirements by providing a JavaFX visual application with core gameplay, authentication, local file processing, Maven configuration, and documentation. The project also demonstrates algorithmic thinking through maze generation and pathfinding, while presenting the result as a polished desktop game.